

Clock Synchronization in Distributed Systems

Lecture 5

Clocks

- How time is really measured?
- Earlier: Solar day, solar second, mean solar second
- Solar day: time between two consecutive transits of the sun
- Solar second: $1/86400$ of a solar day
- Mean solar day: average length of a solar day
- Problem: solar day gets longer because of slowdown of earth's rotation (300 million years ago there were 400 days per year)

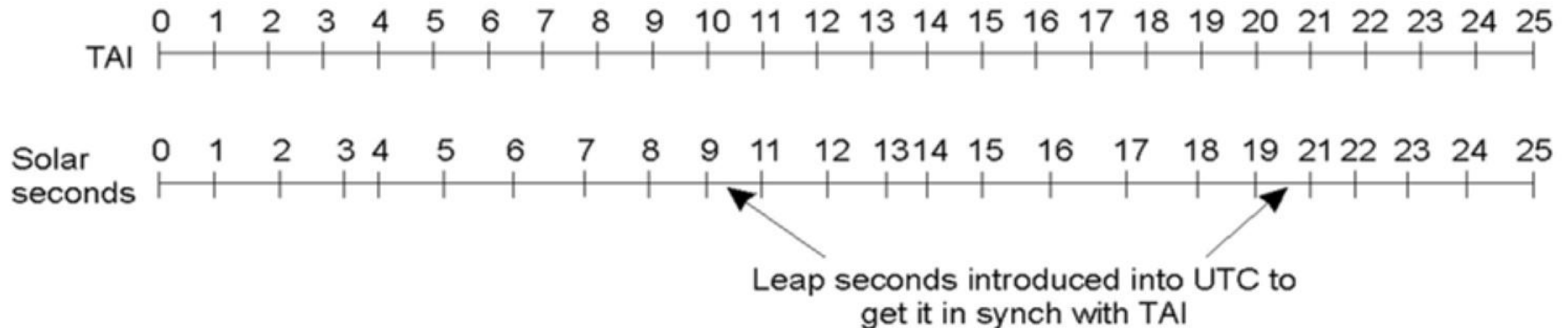
Physical Clocks

- Measures the actual real-world time
- International Atomic Time (TAI): Number of ticks of Cesium 133 atom since 1/1/58 (atomic second).
- Atomic Clock: One second is defined as (since 1967) 9,192,631,770 transitions of the atom Cesium 133.

Physical Clocks

- Because of slowdown of earth, leap seconds have to be introduced
- Correction of TAI is called Universal Coordinated Time (UTC): 30 leap seconds introduced so far
- Network Time Protocol (NTP) can synchronize globally with an accuracy of up to 50 msec

Physical Clocks



- TAI seconds are of constant length, unlike solar seconds. Leap seconds are introduced when necessary to keep in phase with the sun.

Why is Clock Synchronization Important?

- Distributed systems rely on synchronized clocks for consistency.
- Ensures correct event ordering and coordination.
- Applications: Databases, File Systems, Real-Time Systems.

Why synchronization?

You want to catch the 5pm train at the Fort railway station,
but your watch is off by 15 minutes

What if your watch is Late by 15 minutes?

What if your watch is Fast by 15 minutes?

Synchronization is required for

Correctness

Fairness

Why synchronization?

- `make` recompiles if *foo.c* is newer than *foo.o*
- Scenario
 - `make` on machine *A* to build *foo.o*
 - Test on machine *B*; find and fix a bug in *foo.c*
 - Re-run `make` on machine *B*
 - *Nothing happens!*
- Why?

Why synchronization?

Airline reservation system

Server A receives a client request to purchase last ticket on flight ABC 123.

Server A timestamps purchase using local clock **9h:15m:32.45s**, and logs it. Replies ok to client.

That was the last seat. Server A sends message to Server B saying “flight full.”

B enters “Flight ABC 123 full” + local clock value (which reads **9h:10m:10.11s**) into its log.

Server C queries A's and B's logs. Is confused that a client purchased a ticket after the flight became full.

May execute incorrect or unfair actions.

Physical Clocks & Synchronization

In a DS, each process has its own clock.

Clock Skew versus Drift

- Clock **Skew** = Relative Difference in clock *values* of two processes
- Clock **Drift** = Relative Difference in clock *frequencies (rates)* of two processes

A non-zero clock drift causes skew to increase (eventually).

Maximum Drift Rate (**MDR**) of a clock

Absolute MDR is defined relative to Coordinated Universal Time (UTC).

UTC is the “correct” time at any point of time.

MDR of a process depends on the environment.

Effects of Clock Skew in Distributed Systems

- Event ordering inconsistencies.
- Incorrect timestamps in logs and transactions.
- Challenges in leader election and consensus protocols.
- Data corruption in distributed databases.

Sources of Clock Drift in Computers

- Manufacturing differences in hardware.
- Environmental factors such as temperature changes.
- Power fluctuations affecting oscillator stability.

Basics – Processes and Events

An Asynchronous Distributed System (DS) consists of a number of *processes*.

Each process has a *state* (values of variables).

Each process takes *actions* to change its state, which may be an *instruction* or a communication action (*send*, *receive*).

An *event* is the occurrence of an action.

Each process has a local clock – events *within* a process can be assigned *timestamps*, and thus ordered linearly.

But – in a DS, we also need to know the time order of events across different processes.

- ☹ Clocks across processes are not synchronized in an asynchronous DS
unlike in a multicore system

Challenges in Synchronizing Clocks

- Hardware clocks drift over time.
- Network latency affects time accuracy.
- No global clock in distributed environments.
- Message transmission delays are unpredictable.

Synchronizing Physical Clocks

$C_i(t)$: the reading of the software clock at process i when the real time is t .

External synchronization: For a synchronization bound $D > 0$, and for source S of UTC time,

$$|S(t) - C_i(t)| < D,$$

for $i=1,2,\dots,N$ and for all real times t .

Clocks C_i are externally accurate to within the bound D .

In external synchronization, clock is synchronized with an authoritative external source of time

Synchronizing Physical Clocks

Internal synchronization: For a synchronization bound $D > 0$,

$$|C_i(t) - C_j(t)| < D$$

for $i, j = 1, 2, \dots, N$ and for all real times t .

Clocks C_i are internally accurate within the bound D .

In internal synchronization clocks are synchronized with one another with a known degree of accuracy

External synchronization with $D \Rightarrow$ Internal synchronization with $2D$

Internal synchronization with $D \Rightarrow$ External synchronization with ??

Time Synchronization Models

Time Sync Model	Message Delay Behavior
Fully Synchronous (mostly theoretical abstraction)	Fixed, known delay bounds
Partially Synchronous	Variable, but eventually bounded
Fully Asynchronous	Unbounded, unpredictable

- **Note:** Time-synchronization models define timing assumptions of a system, whereas synchronous/asynchronous communication describes message-passing behavior. These are independent concepts.

External Clock Synchronization

- UTC signals are synchronized and broadcast regularly from land-based radio stations and satellites covering many parts of the world.
- Example (US): The radio station WWV broadcasts time signals on several short-wave frequencies.
- Satellite sources include Geo-stationary Operational Environmental Satellites (GOES) and GPS.

External Synchronization – propagation delay

- Radio waves travel near the speed of light. The propagation delay can be accounted for if the exact speed and distance from the source are known.
- Propagation speed varies with atmospheric conditions, leading to inaccuracies.
- Accuracy of a received signal depends on the accuracy of the source and the distance from the source through the atmosphere.

External Synchronization: Clock Drift and Synchronization Interval

- The constant ρ is specified by the manufacturer and is known as the maximum drift rate.
- If two clocks are drifting from Universal Coordinated Time (UTC) in opposite directions, at time Δt after they are synchronized, they may be as much as $2 * \rho * \Delta t$ apart.
- - If the operating system designer wants to guarantee that no two clocks ever differ by more than δ , clocks must be synchronized at least every $\delta / 2\rho$ seconds.

Question

- Given:
- $\rho = 10^{-5}$ (drift rate)
- $\delta = 1$ ms (maximum allowed difference)
- Formula:
- $T \leq \delta / (2\rho)$
- Find:
- How often must clocks synchronize?

Calculation

- $T \leq \delta / (2\rho)$
- $T \leq 0.001 / (2 \times 10^{-5})$
- $T \leq 0.001 / 0.00002$
- $T \leq 50$ seconds

Conceptual Insight

- $T \propto \delta / \rho$
- If drift increases ($\rho \uparrow$):
 - $\rightarrow$ Sync more often
- If tolerance decreases ($\delta \downarrow$):
 - $\rightarrow$ Sync more often

External Synchronization - Algorithms

- NTP – Network Time Protocol – Used for general-purpose time synchronizations (e.g. web servers)
- PTP – Precision Time Protocol – Used for applications that need ultra precise time synchronization (e.g. stock trading platforms)
- Partially synchronous (message delays are bounded)

The Network Time Protocol (NTP)

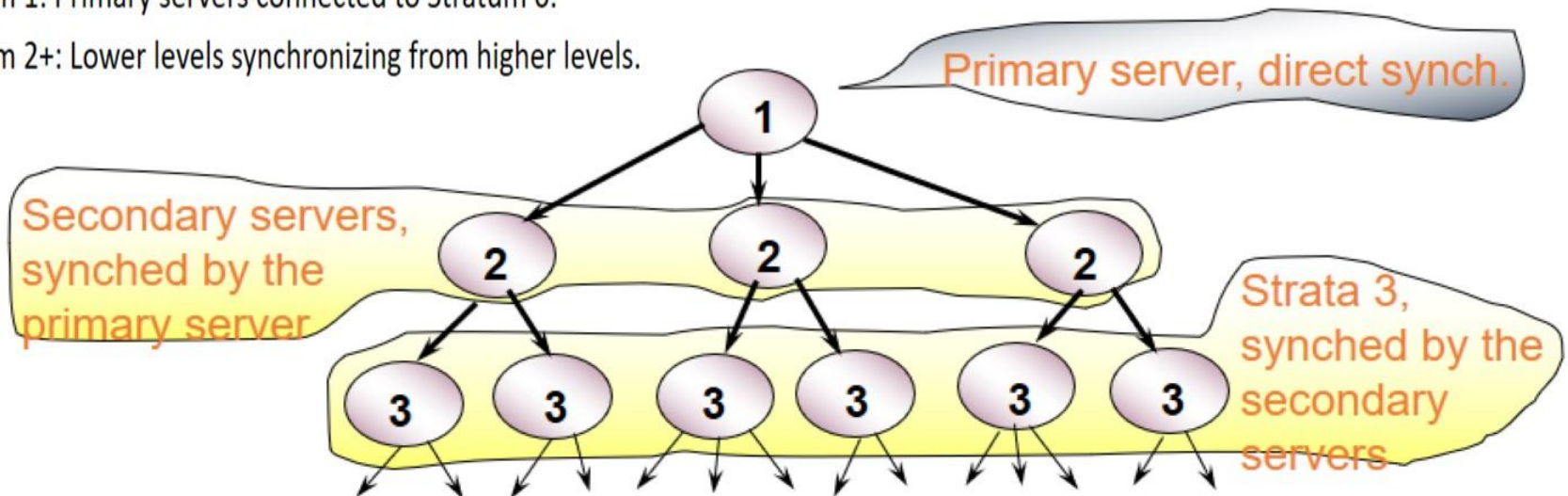
- NTP defines an architecture for a time service and a protocol to distribute time information over the Internet:
- Provides a service enabling clients across the Internet to be synchronized accurately to UTC.
- Provides a reliable service that can survive lengthy losses of connectivity.
- Enables clients to resynchronize sufficiently frequently to offset clock drifts.
- Provides protection against interference with the time service.

The Network Time Protocol (NTP)

Uses a network of time servers to synchronize all processes on a network.

Time servers are connected by a synchronization subnet tree. The root is in touch with UTC. Each node synchronizes its children nodes.

- Stratum 0: Atomic clocks, GPS.
- Stratum 1: Primary servers connected to Stratum 0.
- Stratum 2+: Lower levels synchronizing from higher levels.



Messages exchanged between a pair of NTP peers

- Three modes of synchronization
- Multicast
 - Intended for use in high-speed LAN
 - One or more servers periodically multicasts the time to servers running in the other computers connected to LAN
 - Servers set their clocks assuming a small delay
 - Relatively low accuracies – considered sufficient for many purposes

Messages exchanged between a pair of NTP peers

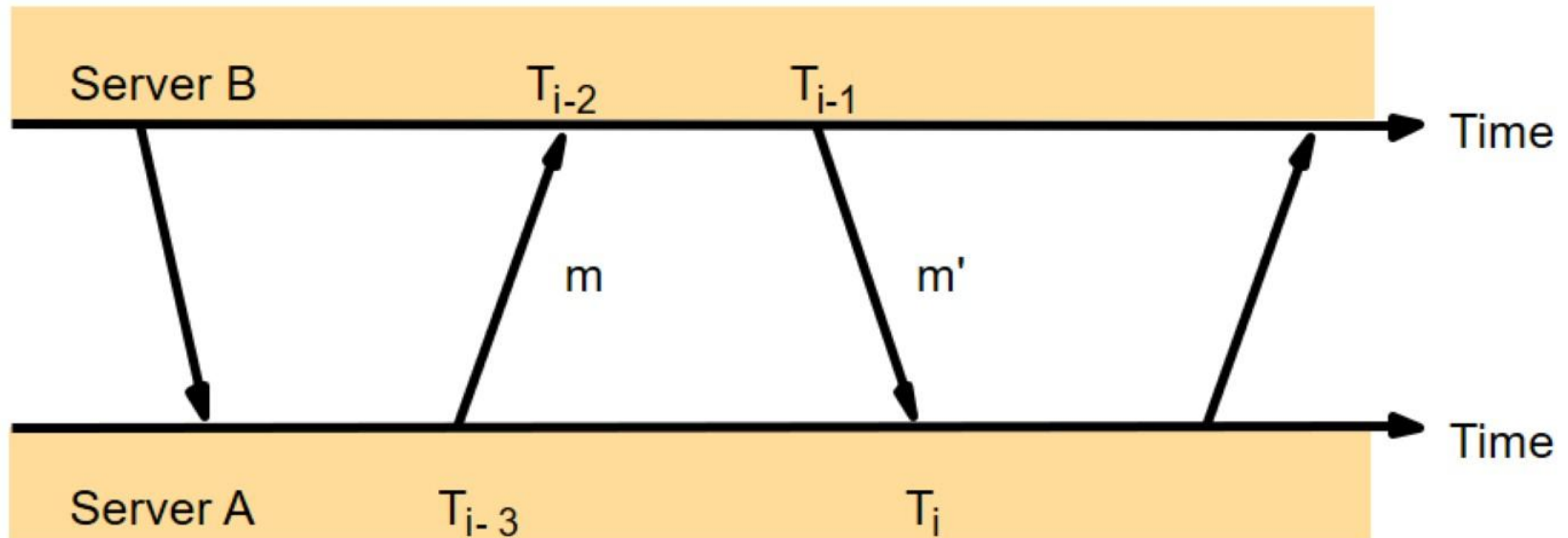
- **Procedure-call mode**

- Similar to the operation of the Cristian's Algorithm
- One server accepts requests from other servers, which it processes by replying with its timestamp
- Suitable where higher accuracies are required than can be achieved with multicast or where multicast is not supported in hardware
- E.g. file servers on the same or another LAN could contact a local server in procedure-call mode

Messages exchanged between a pair of NTP peers

- Symmetric mode
 - Intended for use by the servers that supply time information in LANs and by the higher levels (lower strata) of the synchronization subnet, where the highest accuracies to be achieved
 - A pair of servers operating in symmetric mode exchange messages bearing timing information

Messages Exchanged Between a Pair of NTP Peers ("Connected Servers")



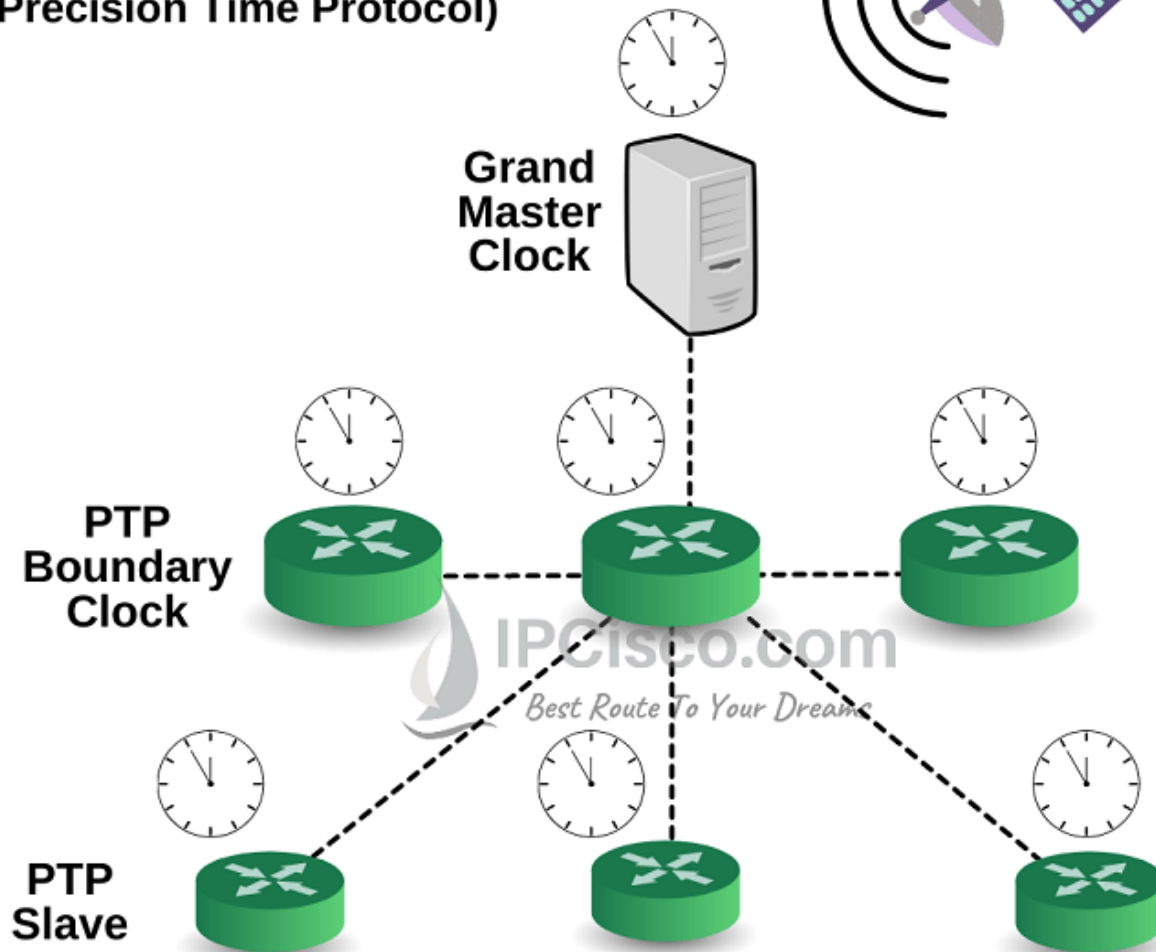
Each message bears timestamps of recent message events: the local time when the previous NTP message was sent and received, and the local time when the current message was transmitted.

What is PTP?

- Precision Time Protocol (PTP) is a high-accuracy time synchronization protocol defined in IEEE 1588.
- It provides sub-microsecond precision, making it ideal for telecom, finance, automation, and data centers.
- Uses hardware timestamping to reduce synchronization errors.
- Works in a hierarchical structure with Grandmaster, Boundary, and Transparent Clocks.

PTP Device Types

(Precision Time Protocol)



How PTP Works

1. Grandmaster Clock sends a Sync message (T1).
2. The Slave records the received time (T2).
3. The Slave Clock sends a Delay Request (T3) to measure the round-trip time.
4. Grandmaster responds with a Delay Response (T4).
5. Using T1, T2, T3, T4, the Slave Clock calculates offset and delay, adjusting its time accordingly.

PTP vs. NTP

Feature	PTP (Precision Time Protocol)	NTP (Network Time Protocol)
Accuracy	Sub-microsecond (<1 μ s)	Millisecond level
Method	Hardware timestamping	Software timestamping
Use Case	Telecom, automation, trading	General IT, distributed systems
Hierarchy	Grandmaster, Boundary, Transparent Clocks	Stratum-based (Stratum 0-15)

Compensating for clock drift

- Compare time T_s provided by the time server to time T_c at computer C
 - If $T_s > T_c$ (e.g. 9:07 am vs 9:05 am), could advance C's time to T_s
 - May miss some clock ticks, probably OK
 - If $T_s < T_c$ (e.g. 9:07 am vs 9:10 am), cannot rollback C's time to T_s
 - Many applications assume that time always advances

Compensating for clock drift

- The solution is not to set C's clock back – but can cause C's clock to run slowly until it resynchronizes with the time server
- This can be achieved in SW, w/o changing the rate at which the HW clock ticks (an operation which is not always supported by HW clocks)

Physical clocks - Internal Synchronization

- Internal synchronization ensures that all nodes in a distributed system have consistent relative time.
- Does not require synchronization with UTC.

Internal Clock Synchronization in a synchronous system

- Consider internal synchronization between two processes in a synchronous distributed system.
- **P** sends time **t** on its local clock to **Q** in a message **m**.
- In principle, **Q** could set its clock to **t + T_trans**, where **T_trans** is the transmission time of **m**.
- The two processes would then agree on time (internal synchronization).

Internal Clock Synchronization in a partially synchronous System

- Unfortunately, **T_{trans}** is subject to variation and is unknown.
- All processes are competing for resources with **P** and **Q** , and other messages are competing with **m** for the network.
- But there is always a minimum transmission time **$\min$** that would be obtained if no other processes executed and no other network traffic existed.
- **$\min$** can be measured or conservatively estimated.

Internal Clock Synchronization in a Partially synchronous System

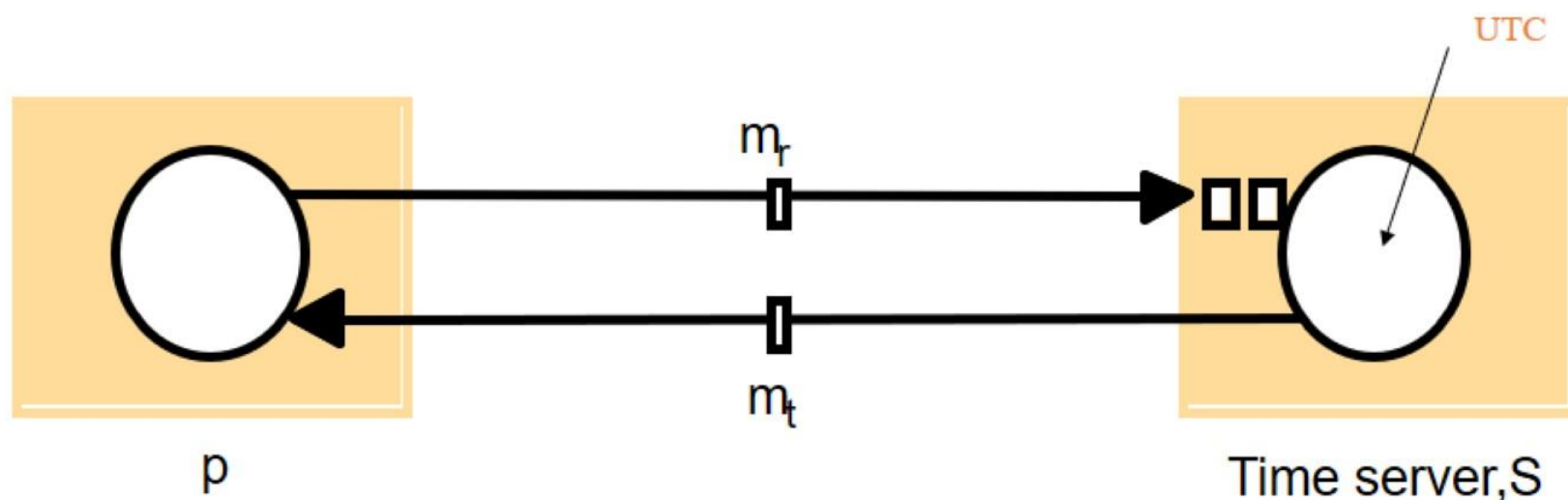
- In a partially synchronous system, there is an upper bound **max** on message transmission time.
- Uncertainty in transmission time: **$u = \text{max} - \text{min}$**
- If **Q** sets its clock to **$(t + \text{min})$** , skew may be as large as **u**.
- If **Q** sets its clock to **$(t + \text{max})$** , skew may again be as large as **u**.
- If **Q** sets its clock to **$(t + (\text{max} - \text{min})/2)$** , the skew is at most **$u/2$** .
- Optimal bound for synchronizing **N** clocks: **$u(1 - 1/N)$** .

Cristian's Clock Synchronization Algorithm – Internal Synchronization

- Introduced by Flaviu Cristian in 1989
- Works in a partially synchronous system (RTT is unbounded but often short in practice)
- Key Observations:
 - **RTT** between client and server is theoretically unbounded, but usually short enough for practical synchronization.
 - Achieves synchronization only if **RTT** is sufficiently short compared to the required accuracy.
 - Practical time estimation is possible if network delays remain predictable within some bound.

Cristian's clock synchronization algorithm

- Periodically ($< \delta/2\rho$) each machine sends a request message m_r and the server sends the time in m_t .
- The roundtrip message transmission delay time is computed, say this is T_{round} .
- The algorithm sets $t + T_{round}/2$ as the new time. (naïve i.e. the elapsed time is split equally before & after S placed *time* in m_t .)



Cristian's Algorithm (Accuracy)

- Assumptions
 - Request & reply via same network
 - The value of minimum transmission time *min* is known or conservatively estimated.

Cristian's Algorithm (Accuracy)

- The earliest point at which S could have placed the time in m_t was *min* after p dispatched m_r .
- The latest point at which it could have done this was *min* before m_t arrived at p.
- The time by S's clock when the reply message arrives is therefore in the range $[t + \textit{min}, t + T_{\text{round}} - \textit{min}]$
- The width of this range is $T_{\text{round}} - 2\textit{min}$, so the accuracy is $\pm (T_{\text{round}} / 2 - \textit{min})$

Cristian's Algorithm (Discussion)

- Variability can be dealt with making several requests & taking the min value of RTT to give the accurate estimate
- Cristian suggested making a series of measurements, and throw away measurements which exceed some threshold value. He assumes that these discarded entries are victims of network congestion. These measurements are then averaged to give a better number for network delay and got added to the current time.
- Cristian stated that the message that came back the fastest can be taken to be the most accurate since it presumably encountered the least traffic.

Cristian's Algorithm: Discussion & Limitations

- Strengths:
 - Simple, effective in a client-server setup.
 - Works well in deterministic LAN environments.
- Limitations:
 - Relies on a single server, creating a single point of failure.
 - Assumes network delays are symmetric, which may not always hold.
- Challenges:
 - If the time server fails, redundancy is required (e.g., using multiple servers & Byzantine agreement to resolve conflicts).
 - Susceptible to false clock readings from imposters—requires authentication mechanisms.

The Berkeley Algorithm (internal synchronization)

- Gusella and Zatti at the University of California, Berkeley in 1989.
- A coordinator (time server): *master*
- Just the opposite approach of Cristian's algorithm
 - Periodically the *master* polls the time of each client (slave) whose clocks are to be synchronized.
 - Based on the answer (by observing the RTT as in Cristian's algorithm), it computes the average (including its own clock value) and broadcasts the new time.
- This method is suitable for a system in which no machine has a WWV receiver getting the UTC.
- The coordinator's time must be set manually by the operator periodically.

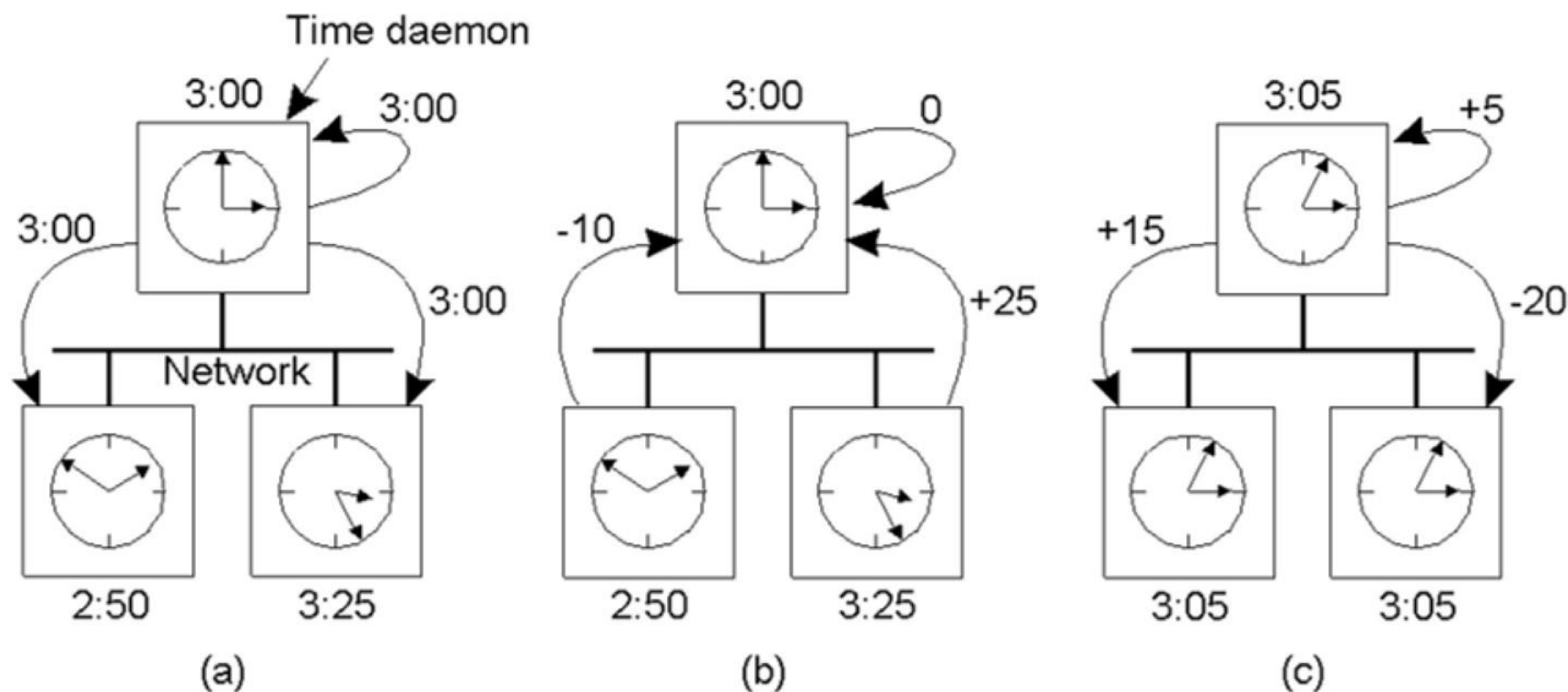
The Berkeley Algorithm

- Berkeley's Algorithm operates in a partially synchronous system where message delays are variable but eventually bounded.
- The balance of probabilities ensures that the average cancels out individual clock drifts.
- The accuracy of synchronization depends on a nominal maximum RTT (Round Trip Time) between the master and the slaves.
- The master eliminates occasional readings that exceed this maximum delay threshold.

The Berkeley Algorithm

- Instead of sending the updated current time back to the comps – which will introduce further uncertainty due to message transmission time – the master send the amount by which each individual slave's clock requires adjustment (+ or -)
- The algorithm eliminates readings from faulty clocks (since these could have significant adverse effects if an ordinary average was taken) – a subset of clock is chosen that do not differ by more than a specified amount and then the average is taken.

The Berkeley Algorithm



- The time daemon asks all the other machines for their clock values
- The machines answer
- The time daemon tells everyone how to adjust their clock

Decentralized Clock Synchronization

- Cristian's and Berkeley's methods are highly centralized, with the usual disadvantages:
 - Single point of failure
 - Congestion around the server
- Decentralized synchronization algorithms divide time into fixed-length resynchronization intervals (e.g. **Gossip based clock sync, Gradient Time Synchronization Protocol – GTSP**)
- -One approach: The i^{th} interval starts at $T_0 + iR$ and runs until $T_0 + (i+1)R$, where:
 - T_0 is an agreed-upon moment in the past.
 - R is a system parameter defining the interval length.

Decentralized Clock Synchronization

- At the beginning of each interval, every machine broadcasts the current time according to its clock.
- After a machine broadcasts its time, it starts a local timer to collect all other broadcasts that arrive during some interval **S**.
- When all broadcasts arrive, an algorithm is run to compute a new synchronized time.

Decentralized Synchronization algorithms

- Some algorithms average out the time across all nodes.
- Discard the m highest and m lowest values, then average the rest—this prevents up to m faulty clocks from influencing synchronization.
- Correct each message by adding an estimate of propagation time from the source.
- This estimate can be made from:
 - The known network topology
 - Timing how long it takes for a probe message to be echoed

Physical Clocks – Clock Synchronization

- External Synchronization - **NTP, PTP**
- Internal Synchronization - Christian's, Berkely, Distributed

Is It Enough to Synchronize Physical Clocks?

- Value received from UTC receivers is only accurate to within 0.1–10 milliseconds.
- At best, we can synchronize clocks to within **10–30 milliseconds** of each other (for **NTP**).
- Frequent synchronization is needed to avoid local clock drift.
- In **10 ms**, a **100 MIPS** machine can execute **1 million** instructions.
- Sufficient for time-of-day tracking but not enough for precise event ordering in distributed systems.
- Cannot reliably determine the relative order of events on different computers.

Limitations of Physical Synchronization

- Network-induced errors.
- Failures in time servers.
- Gradual drift still occurs despite corrections.

Logical Clocks

- Lamport (1978) showed that clock synchronization is possible and presented an algorithm for it. He also pointed out that clock synchronization need not be absolute.
- If two processes do not interact, it is not necessary that their clocks be synchronized because the lack of synchronization would not be observable and thus could not cause problems.

Logical Clocks (Contd)

- Logical clocks do NOT perform synchronization; they only ensure event ordering.
- What usually matters is not that all processes agree on exactly what time it is, but rather, that they agree on the order in which events occur.
- Therefore, it is the internal consistency of the clocks that matters, not whether they are close to real time.
- It is conventional to speak of these as logical clocks rather than physical clocks.

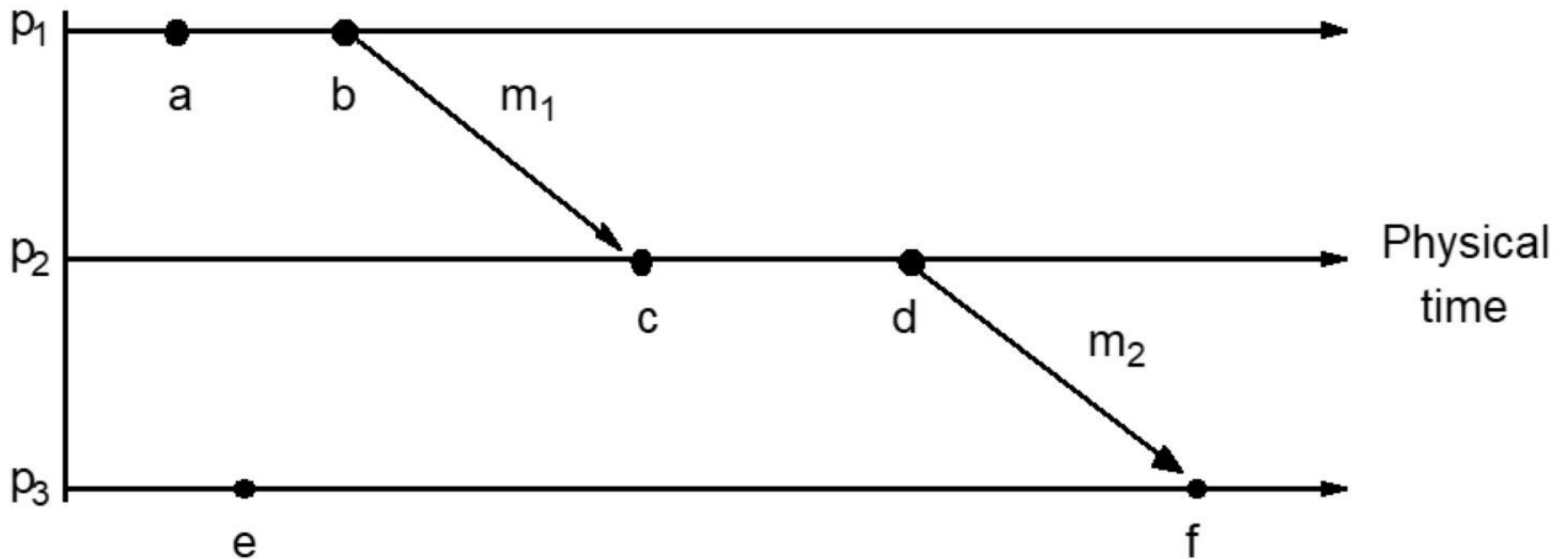
Logical Clocks

- ❖ Is it always necessary to give *absolute* time to events?
- ❖ Suppose we can assign *relative* time to events, in a way that does not violate their *causality*
 - ❖ Well, that would work – we humans run our lives without looking at our watches for everything we do
 - ❖ First proposed by Leslie *Lamport* in the 70's
- ❖ Define a logical relation *Happens-Before* ($\rightarrow$) among events:
 1. On the same process: $a \rightarrow b$, if $time(a) < time(b)$
 2. If p1 sends m to p2: $send(m) \rightarrow receive(m)$
 3. (Transitivity) If $a \rightarrow b$ and $b \rightarrow c$ then $a \rightarrow c$

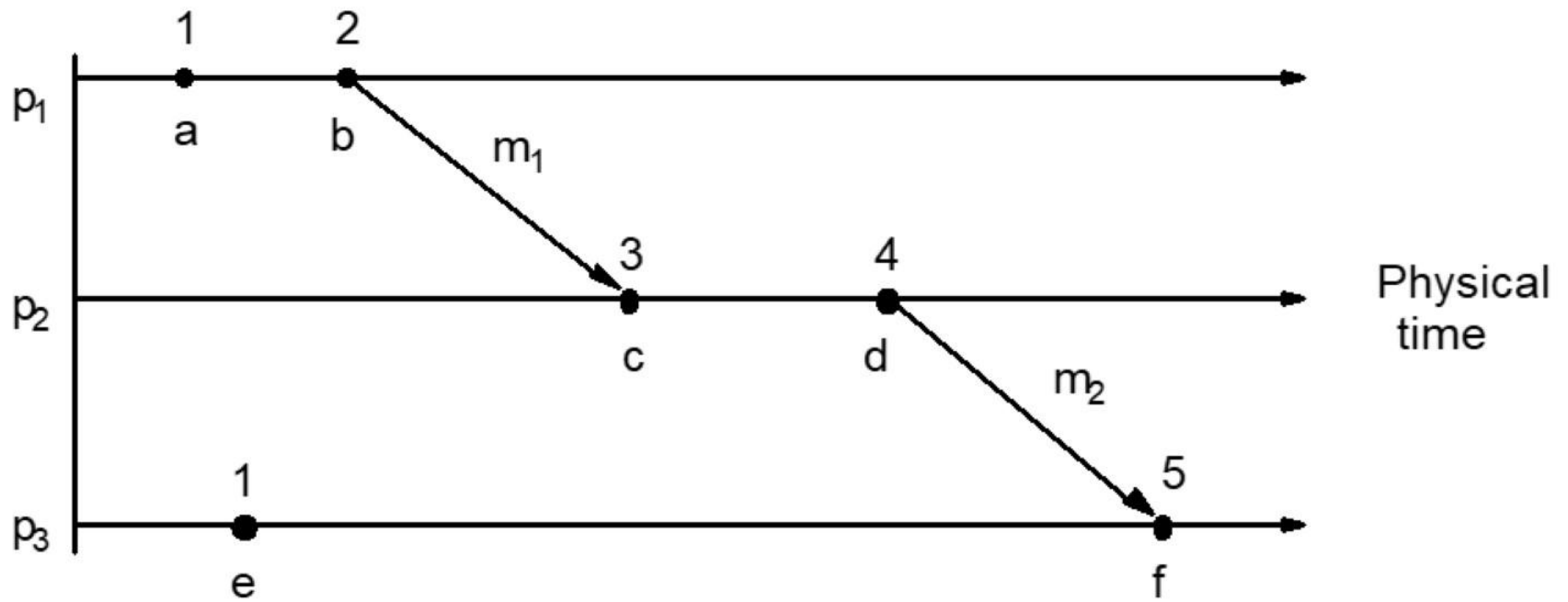
Logical Clocks

- ❖ Lamport Algorithm assigns **logical timestamps to events**:
 - ❑ All processes use a counter (clock) with initial value of zero
 - ❑ A process increments its counter when a **send** or an **instruction** happens at it. The counter is assigned to the event as its timestamp.
 - ❑ A **send (message)** event carries its timestamp
 - ❑ For a **receive (message)** event the counter is updated by $\max(\text{local clock}, \text{message timestamp}) + 1$

Events Occurring at Three Processes

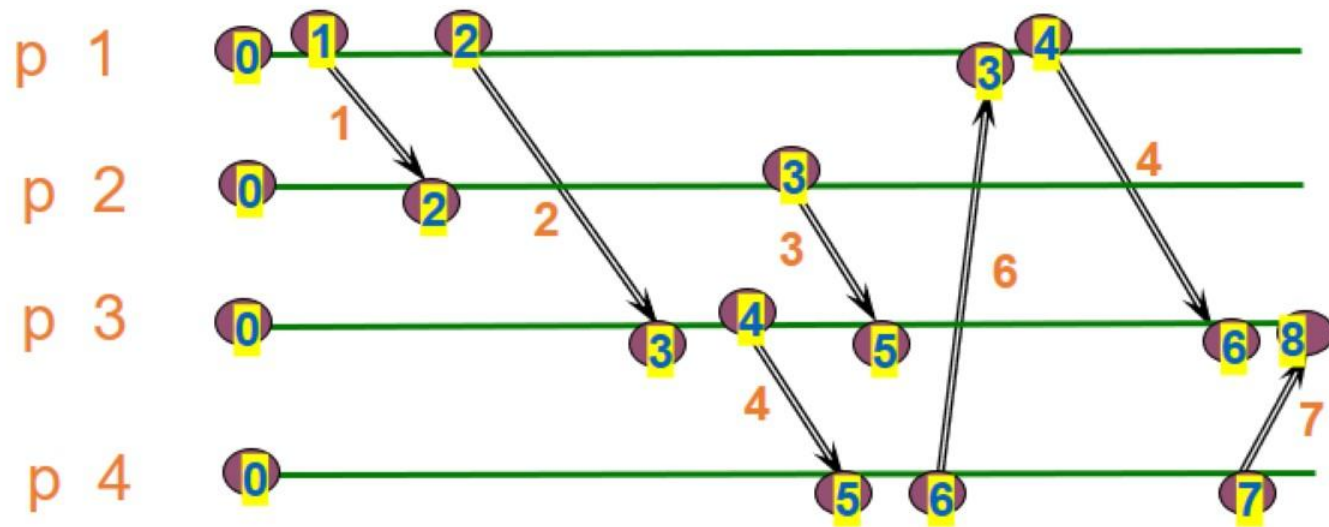


Lamport Timestamps



Find the Mistake: Lamport Logical Time

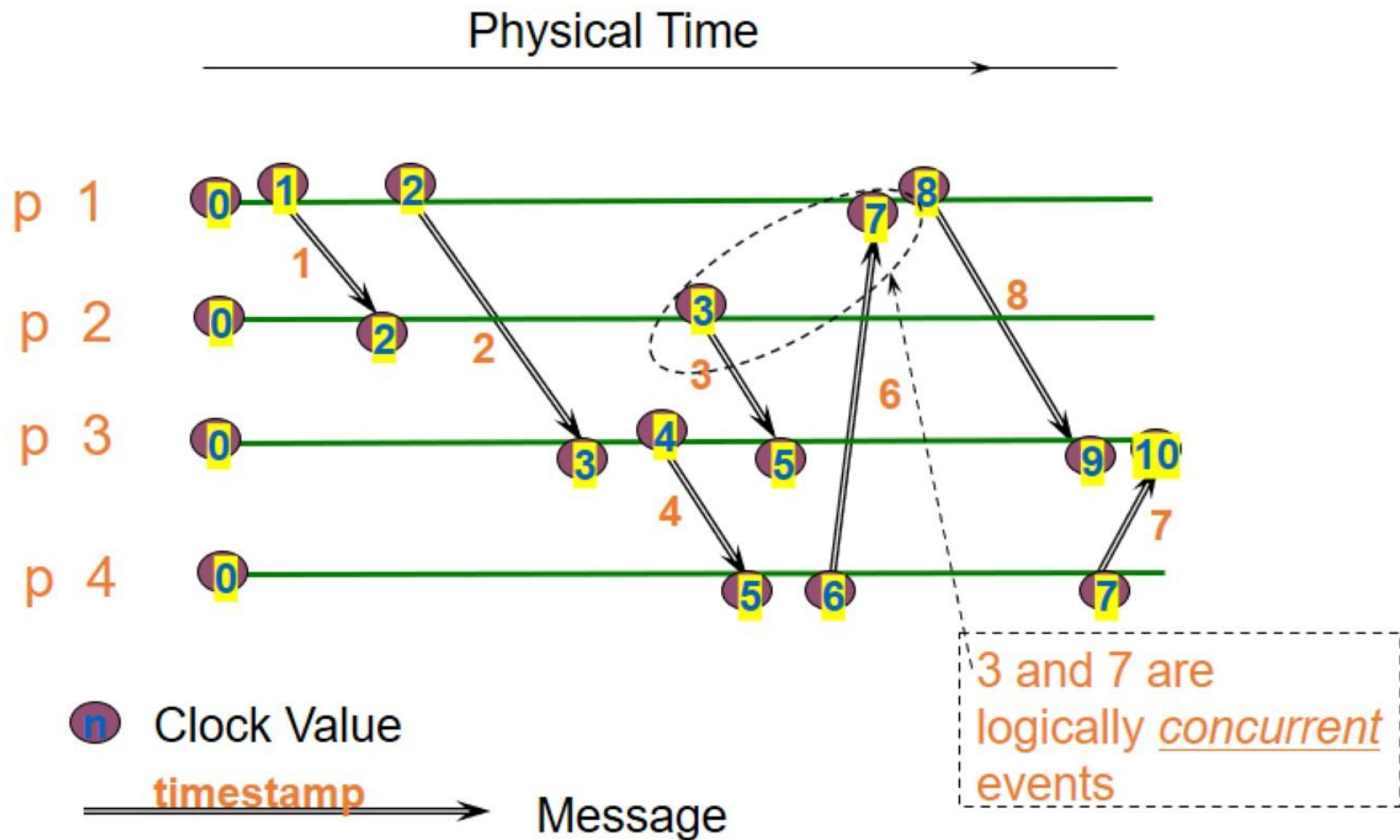
Physical Time →



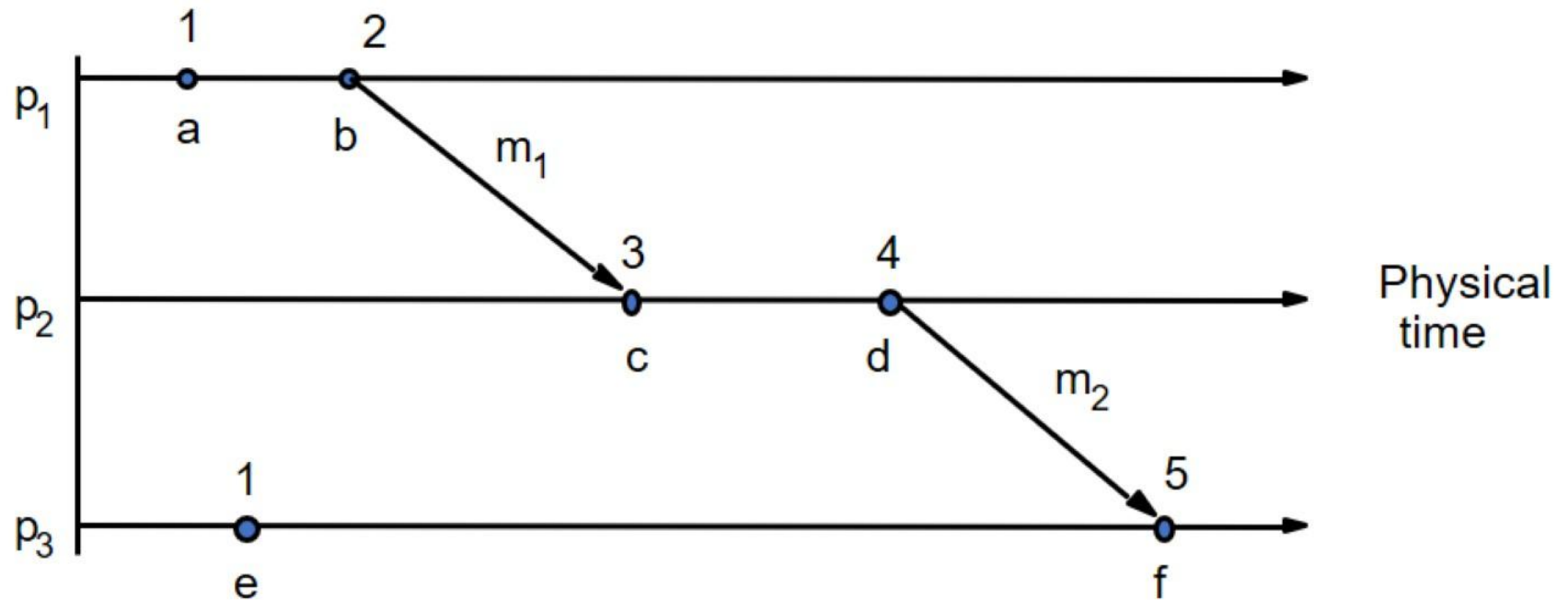
Clock Value

Message

Corrected Example: Lamport Logical Time



Events occurring at three processes



Limitations of Lamport Clocks:

It represents a *partial order*, if $a \rightarrow b$ it implies that $L(a) < L(b)$ but the converse is not true, i.e., if $L(a) < L(b)$ it doesn't imply $a \rightarrow b$. From the fig. $L(b) > L(e)$ but in fact, $b \parallel e$.

Vector Logical Clocks

❖ With Lamport Logical Timestamp

$e \rightarrow f \Rightarrow \text{timestamp}(e) < \text{timestamp}(f)$, but

$\text{timestamp}(e) < \text{timestamp}(f) \Rightarrow \{e \rightarrow f\} \text{ OR } \{e \text{ and } f \text{ concurrent}\}$

❖ Vector Logical time addresses this issue:

- N processes. Each uses a vector of counters (logical clocks), initially all zero. i^{th} element is the clock value for process i.
- Each process i increments the i^{th} element of its vector upon an **instruction** or **send** event. Vector value is timestamp of the event.
- A **send(message)** event carries its vector timestamp (counter vector)
 - For a **receive(message)** event,

$$V_{\text{receiver}}[j] = \begin{cases} \text{Max}(V_{\text{receiver}}[j], V_{\text{message}}[j]), & \text{if } j \text{ is not self} \\ V_{\text{receiver}}[j] + 1 & \text{otherwise} \end{cases}$$

Vector Clocks (Mattern '89 & Fidge '91)

- To overcome the problems of Lamport Clock → Vector clocks
 - A *vector clock* for a system of N processes is an array of N integers. P_i keeps its VC V_i , to timestamp local events.
 - Vector clock rules:

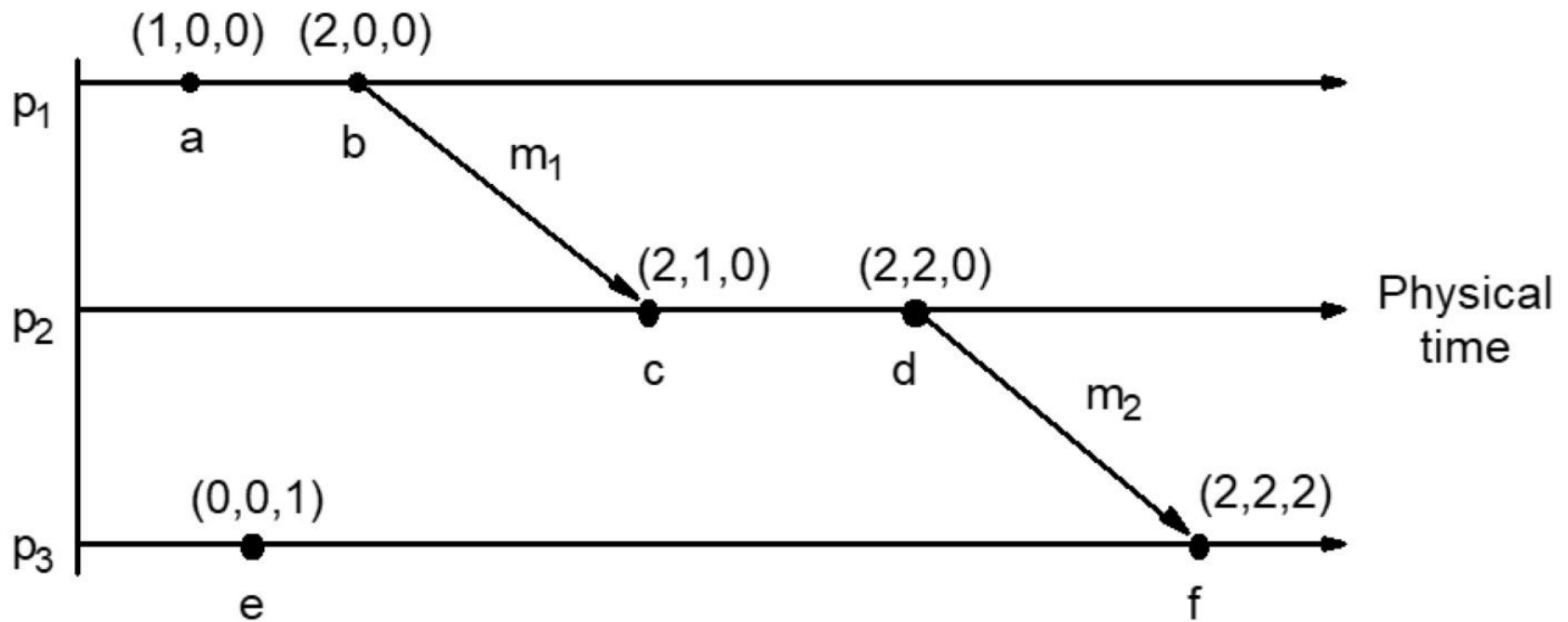
VC1: Initially for each process p_i , $V_i[j] = 0$,
for $j = 1, 2, \dots, N$

VC2: Just before time stamping an event, p_i sets
 $V_i[i] = V_i[i] + 1$.

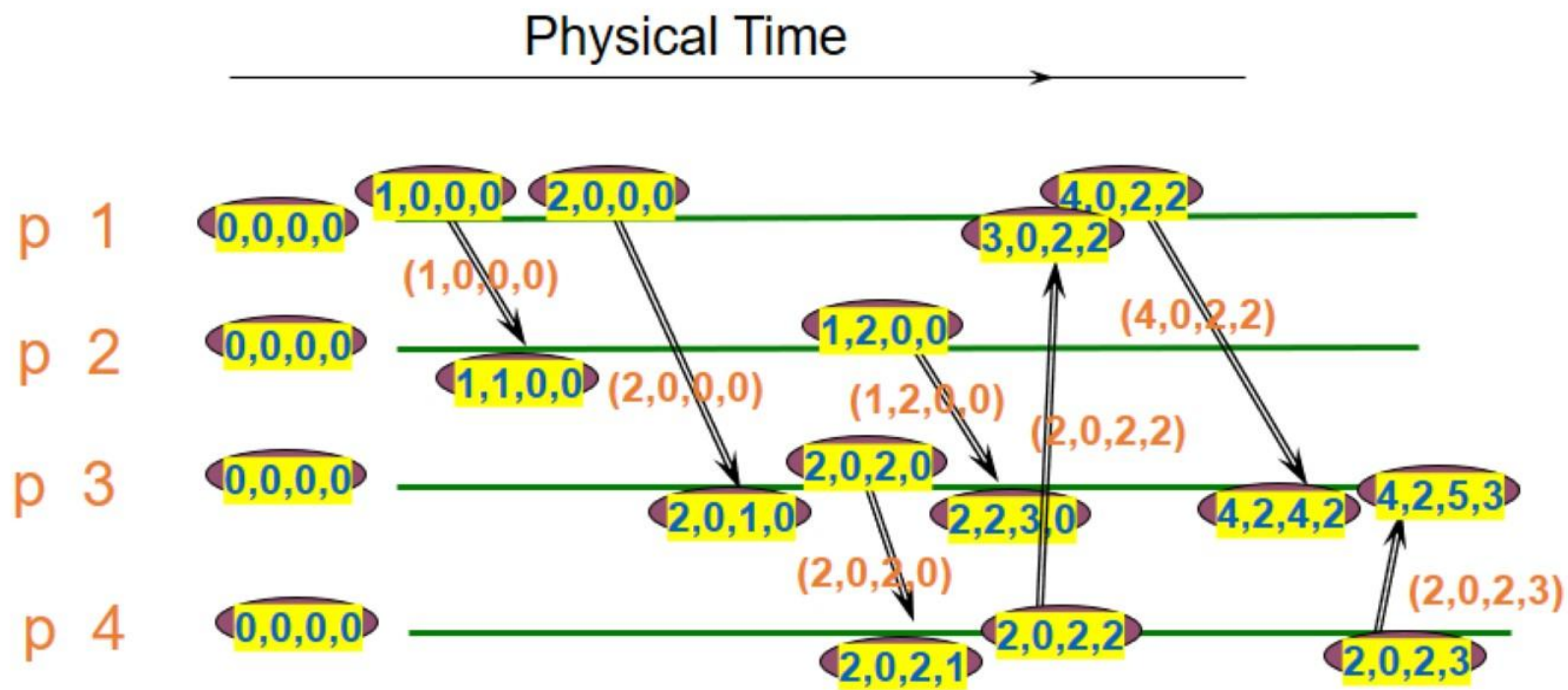
VC3: p_i sends $t = V_i$ in every message it sends

VC4: When p_i receives t in a message, it sets
 $V_i[j] = \max(V_i[j], t[j])$, for $j = 1, 2, \dots, N$
(the max is computed component wise)

Vector Timestamps



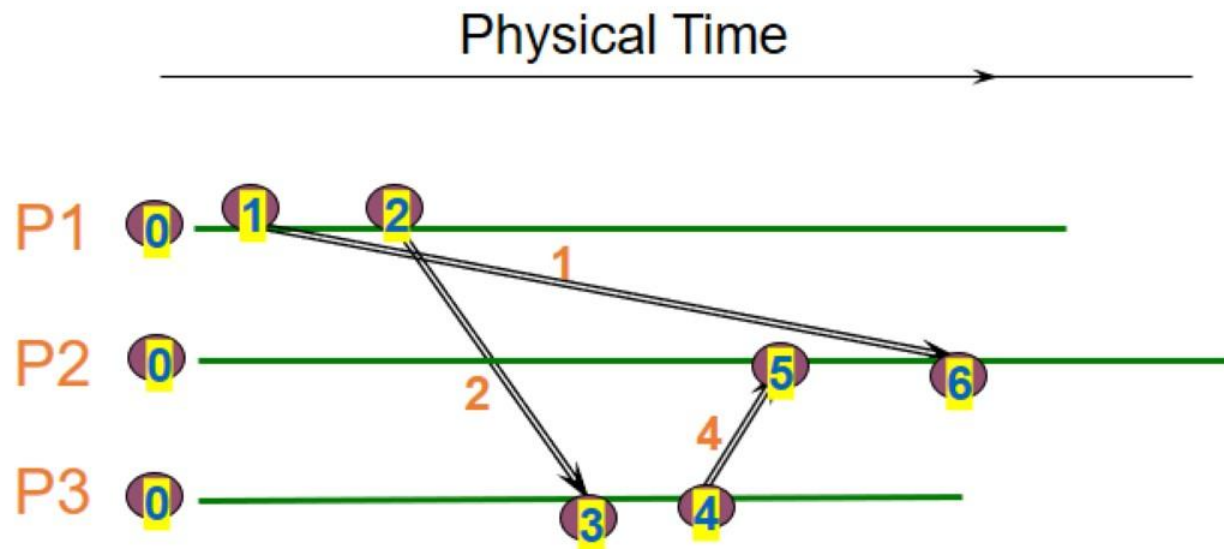
Example: Vector Timestamps



n,m,p,q Vector logical clock

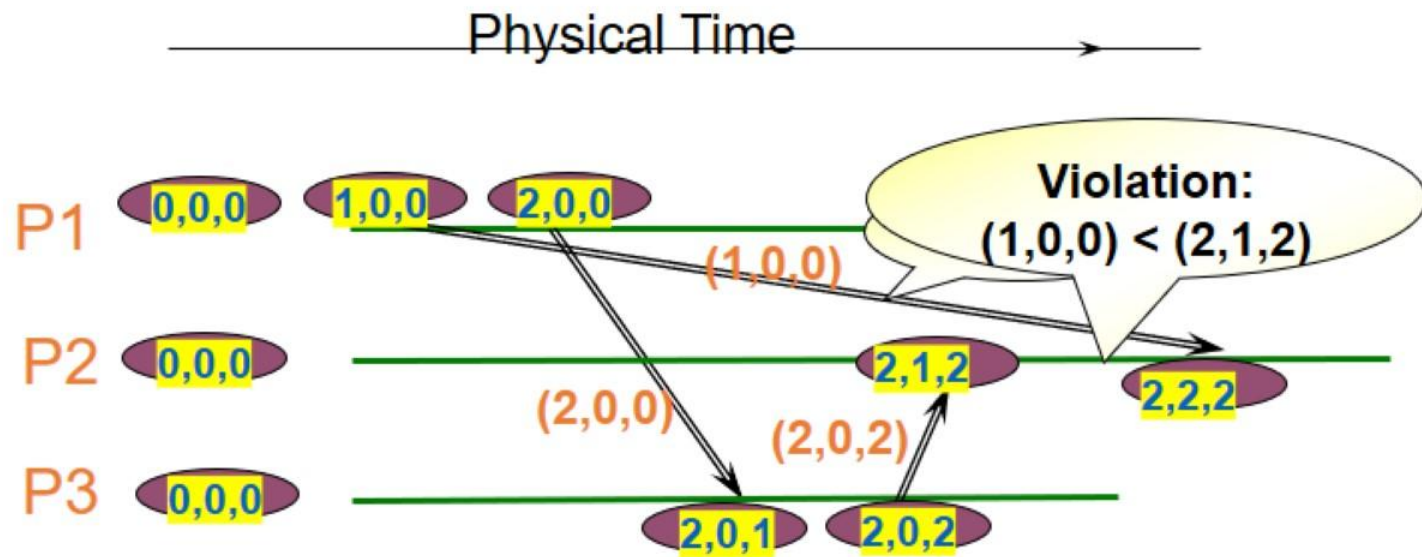
(vector timestamp) → Message

Side Issue: Causality Violation



- Causality violation occurs when order of messages causes an action based on information that another host has not yet received.
- In designing a distributed system, potential for causality violation is important to notice

Detecting Causality Violation



- Potential causality violation can be detected by vector timestamps.
- If the vector timestamp of a message is less than the local vector timestamp, on arrival, there is a potential causality violation.

Clock Synchronization Summary

- Clock synchronization is critical for consistency in distributed systems.
- Physical clocks require frequent resynchronization due to drift and network delays
- External Synchronization (NTP, PTP) aligns clocks with UTC but has accuracy limitations.
- Internal Synchronization (Cristian's, Berkeley) synchronizes clocks within a system but assumes some delay bounds.
- Decentralized Synchronization uses peer-to-peer techniques for scalability.
- Logical Clocks (Lamport, Vector Clocks) focus on event ordering rather than real-time synchronization.
- Challenges include clock drift, message delays, and failure handling.